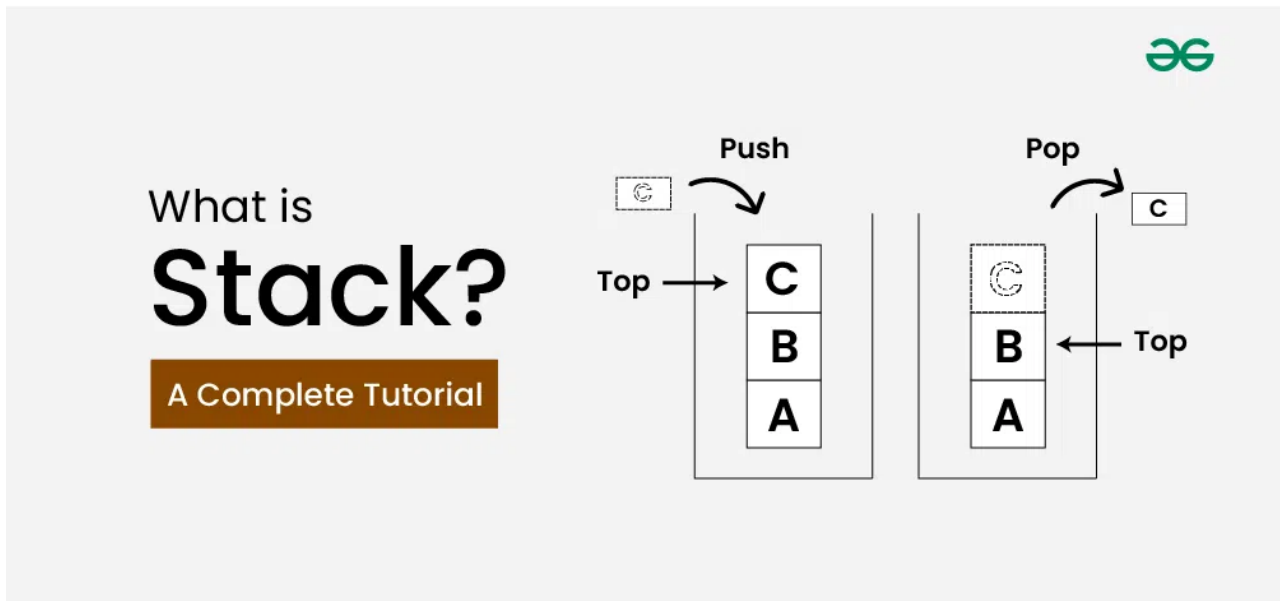


What is Stack Data Structure? A Complete Tutorial

Last Updated : 06 Mar, 2025



Stack is a linear data structure that follows **LIFO (Last In First Out) Principle**, the last element inserted is the first to be popped out. It means both insertion and deletion operations happen at one end only.



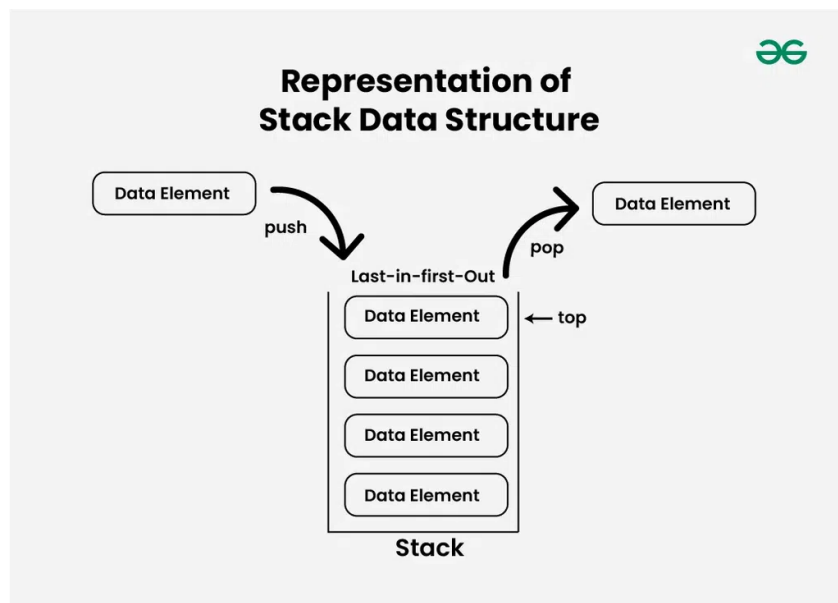
LIFO (Last In First Out) Principle

Here are some real world examples of LIFO

- Consider a stack of plates. When we add a plate, we add at the top. When we remove, we remove from the top.
- A **shuttlecock box** (or any other box that is closed from one end) is another great real-world example of the **LIFO (Last In, First Out)** principle where do insertions and removals from the same end.

Representation of Stack Data Structure:

Stack follows LIFO (Last In First Out) Principle so the element which is pushed last is popped first.



Types of Stack:

- **Fixed Size Stack** : As the name suggests, a fixed size stack has a fixed size and cannot grow or shrink dynamically. If the stack is full and an attempt is made to add an element to it, an overflow error occurs. If the stack is empty and an attempt is made to remove an element from it, an underflow error occurs.
- **Dynamic Size Stack** : A dynamic size stack can grow or shrink dynamically. When the stack is full, it automatically increases its size to accommodate the new element, and when the stack is empty, it decreases its size. This type of stack is implemented using a linked list, as it allows for easy resizing of the stack.

Basic Operations on Stack:

In order to make manipulations in a stack, there are certain operations provided to us.

- **push()** to insert an element into the stack
- **pop()** to remove an element from the stack
- **top()** Returns the top element of the stack.
- **isEmpty()** returns true if stack is empty else false.
- **isFull()** returns true if the stack is full else false.

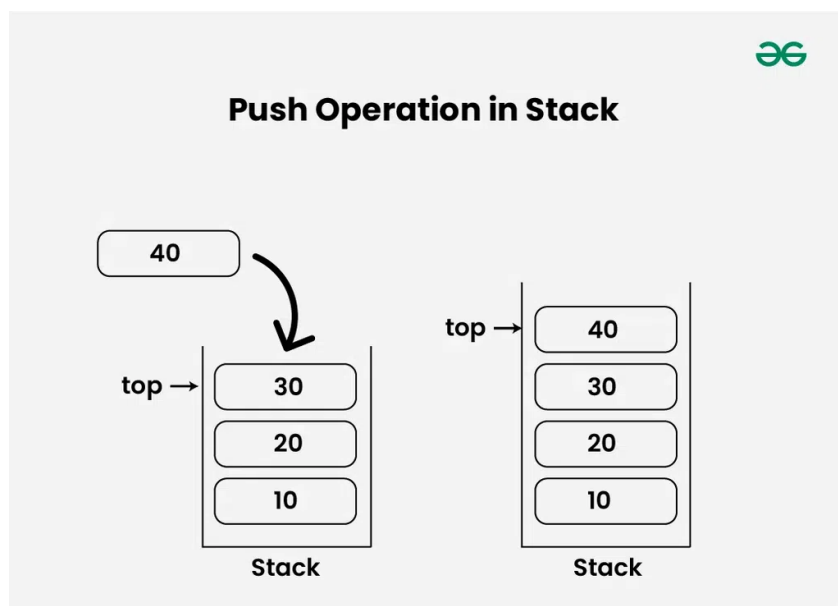
To implement stack, we need to maintain reference to the top item.

Push Operation on Stack

Adds an item to the stack. If the stack is full, then it is said to be an **Overflow condition**.

Algorithm for Push Operation:

- Before pushing the element to the stack, we check if the stack is **full** .
- If the stack is full (**$\text{top} == \text{capacity} - 1$**) , then **Stack Overflows** and we cannot insert the element to the stack.
- Otherwise, we increment the value of top by 1 (**$\text{top} = \text{top} + 1$**) and the new value is inserted at **top position** .
- The elements can be pushed into the stack till we reach the **capacity** of the stack.

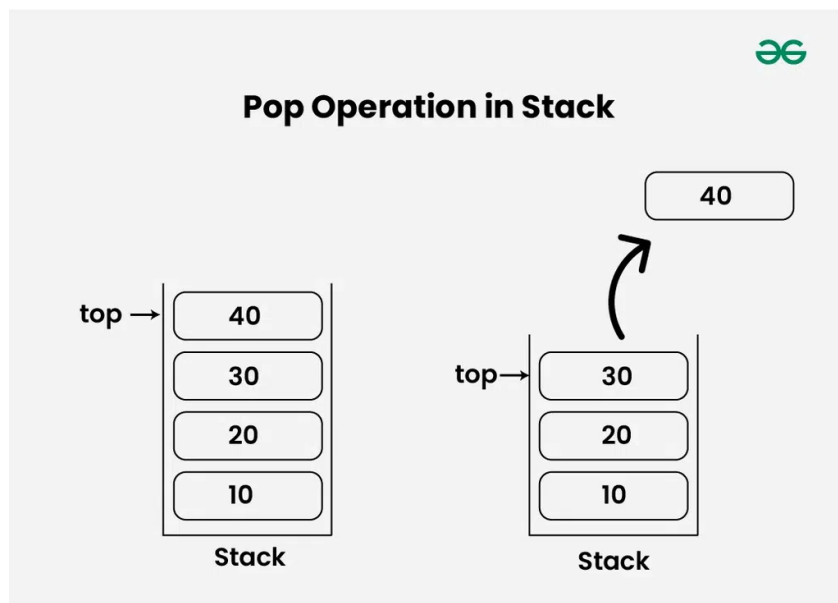


Pop Operation in Stack

Removes an item from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an **Underflow condition**.

Algorithm for Pop Operation:

- Before popping the element from the stack, we check if the stack is **empty** .
- If the stack is empty (**$\text{top} == -1$**), then **Stack Underflows** and we cannot remove any element from the stack.
- Otherwise, we store the value at top, decrement the value of top by 1 (**$\text{top} = \text{top} - 1$**) and return the stored top value.

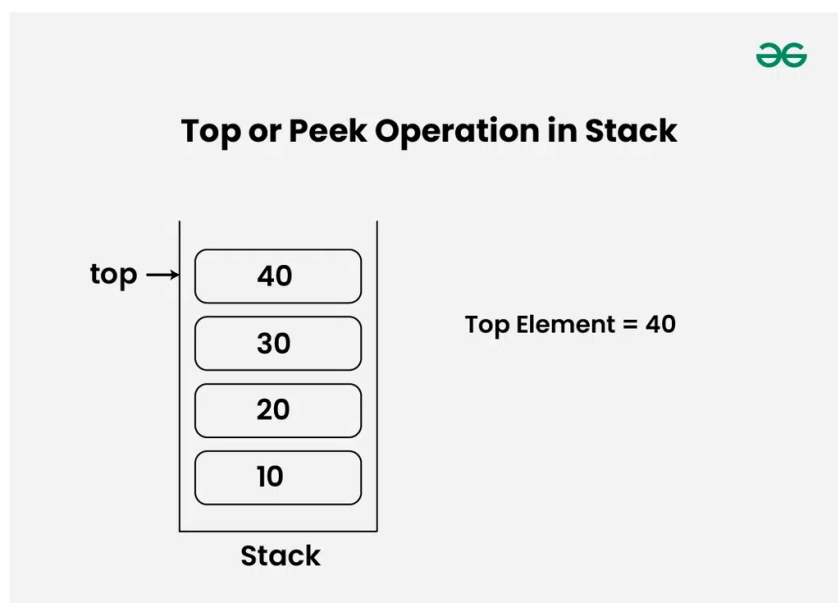


Top or Peek Operation on Stack

Returns the top element of the stack.

Algorithm for Top Operation:

- Before returning the top element from the stack, we check if the stack is empty.
- If the stack is empty ($\text{top} == -1$), we simply print "Stack is empty".
- Otherwise, we return the element stored at **index = top**.

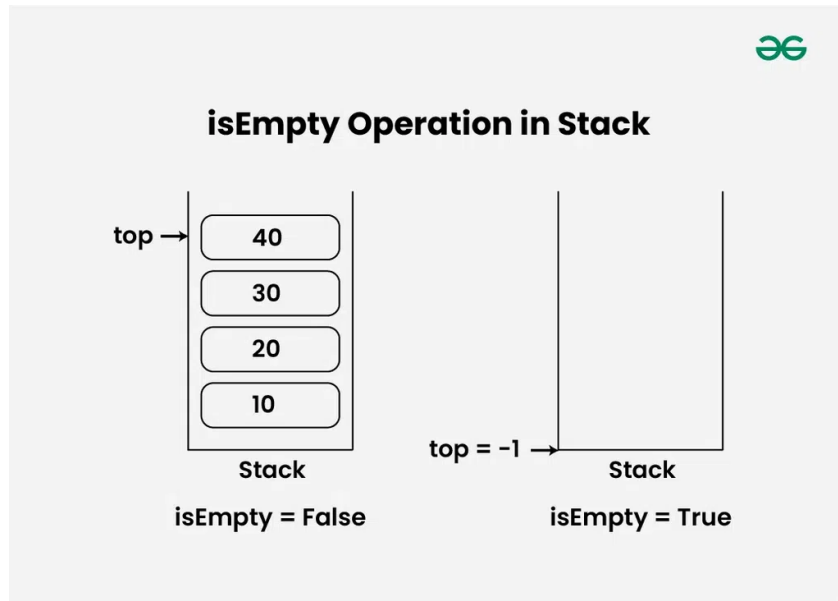


isEmpty Operation in Stack Data Structure:

Returns true if the stack is empty, else false.

Algorithm for isEmpty Operation:

- Check for the value of **top** in stack.
- If (**top == -1**), then the stack is **empty** so return **true** .
- Otherwise, the stack is not empty so return **false** .



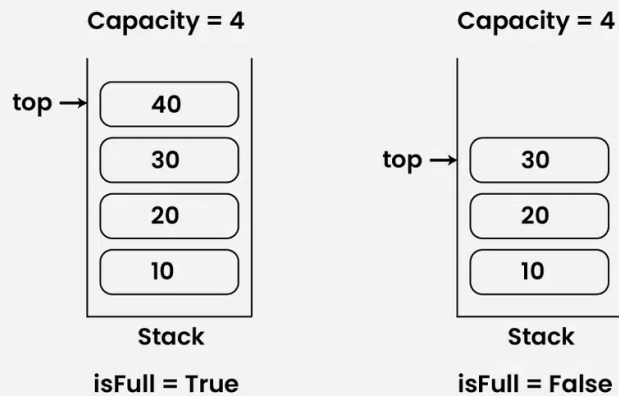
isFull Operation in Stack Data Structure:

Returns true if the stack is full, else false.

Algorithm for isFull Operation:

- Check for the value of **top** in stack.
- If (**top == capacity-1**), then the stack is **full** so return **true**.
- Otherwise, the stack is not full so return **false**.

isFull Operation in Stack



Implementation of Stack

The basic operations that can be performed on a stack include push, pop, and peek. There are two ways to implement a stack -

- [Implementation of Stack using Array](#)
- [Implementation of Stack using Linked List](#)
- [Implementation of Stack using Deque](#)

Complexity Analysis of Operations on Stack Data Structure:

Operations	Time Complexity	Space Complexity
push()	$O(1)$	$O(1)$
pop()	$O(1)$	$O(1)$
top() or peek()	$O(1)$	$O(1)$
isEmpty()	$O(1)$	$O(1)$
isFull()	$O(1)$	$O(1)$

Next Articles:

- [Applications, Advantages and Disadvantages of Stack](#)
- [Implement a stack using singly linked list](#)
- [Basic Operations in Stack Data Structure with Implementations](#)
- [Top 50 Problems on Stack Data Structure asked in SDE Interviews](#)
- [Applications, Advantages and Disadvantages of Stack](#)
- [Stack for Competitive Programming](#)

Stack Data Structure

[Visit Course ↗](#)[Comment](#)[More info ▼](#)[Advertise with us](#)[Next Article >](#)

Applications, Advantages and
Disadvantages of Stack

Similar Reads

Stack Data Structure

A Stack is a linear data structure that follows a particular order in which the operations are performed. The order may be LIFO (Last In First Out) or FILO (First In Last Out). LIFO implies that the element that is...

🕒 3 min read

What is Stack Data Structure? A Complete Tutorial

Stack is a linear data structure that follows LIFO (Last In First Out) Principle, the last element inserted is the first to be popped out. It means both insertion and deletion operations happen at one end only...

🕒 4 min read

Applications, Advantages and Disadvantages of Stack

A stack is a linear data structure in which the insertion of a new element and removal of an existing element takes place at the same end represented as the top of the stack. Stack Data...

🕒 2 min read

Implement a stack using singly linked list

To implement a stack using a singly linked list, we need to ensure that all operations follow the LIFO (Last In, First Out) principle. This means that the most recently added element is always the first one to...

🕒 13 min read

Introduction to Monotonic Stack - Data Structure and Algorithm Tutorials

A monotonic stack is a special data structure used in algorithmic problem-solving. Monotonic Stack maintaining elements in either increasing or decreasing order. It is commonly used to efficiently solve...

🕒 12 min read

Difference Between Stack and Queue Data Structures

In computer science, data structures are fundamental concepts that are crucial for organizing and storing data efficiently. Among the various data structures, stacks and queues are two of the most basic yet...

🕒 4 min read

Stack implementation in different language



Some questions related to Stack implementation



Easy problems on Stack



Intermediate problems on Stack

